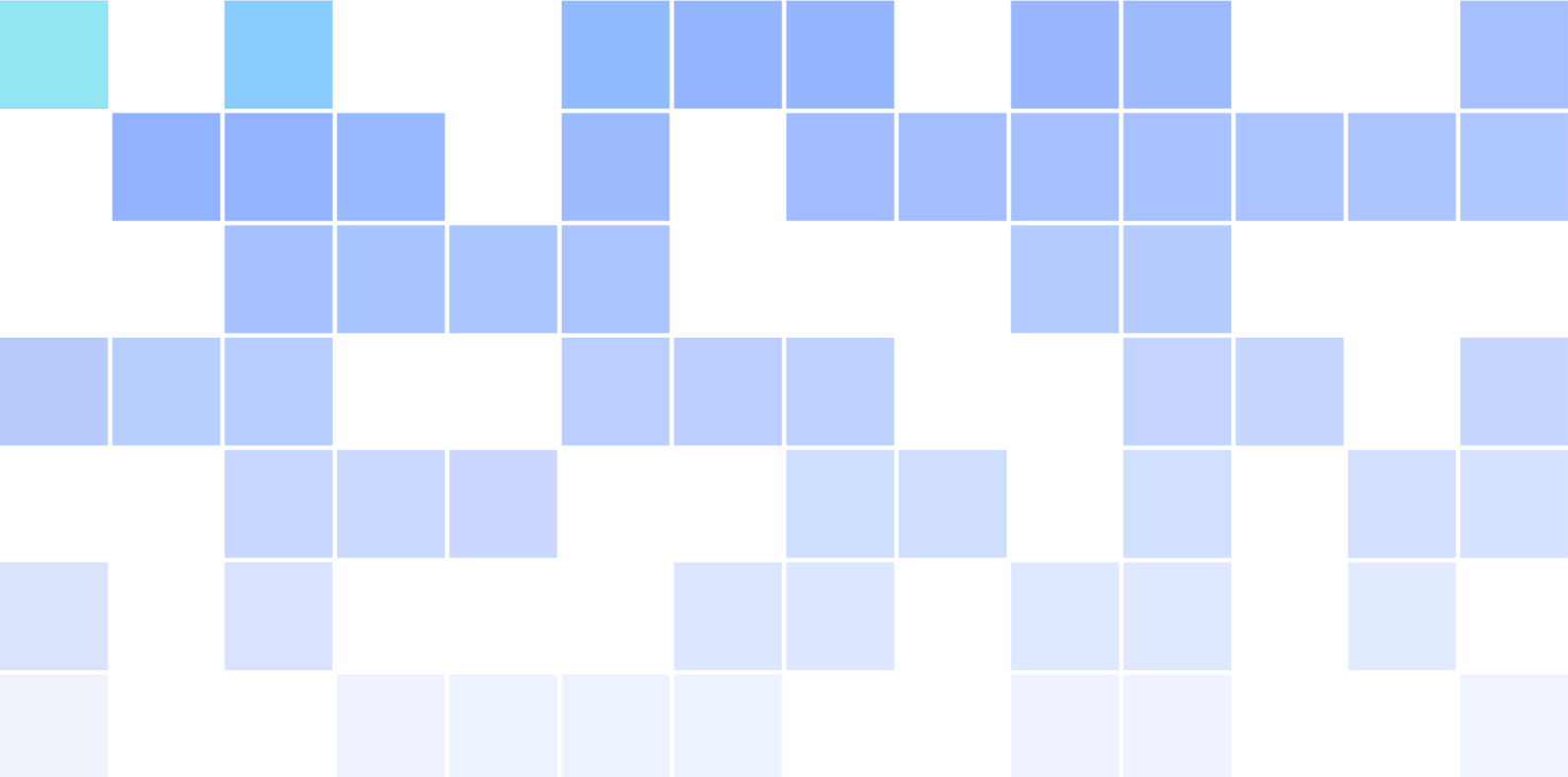


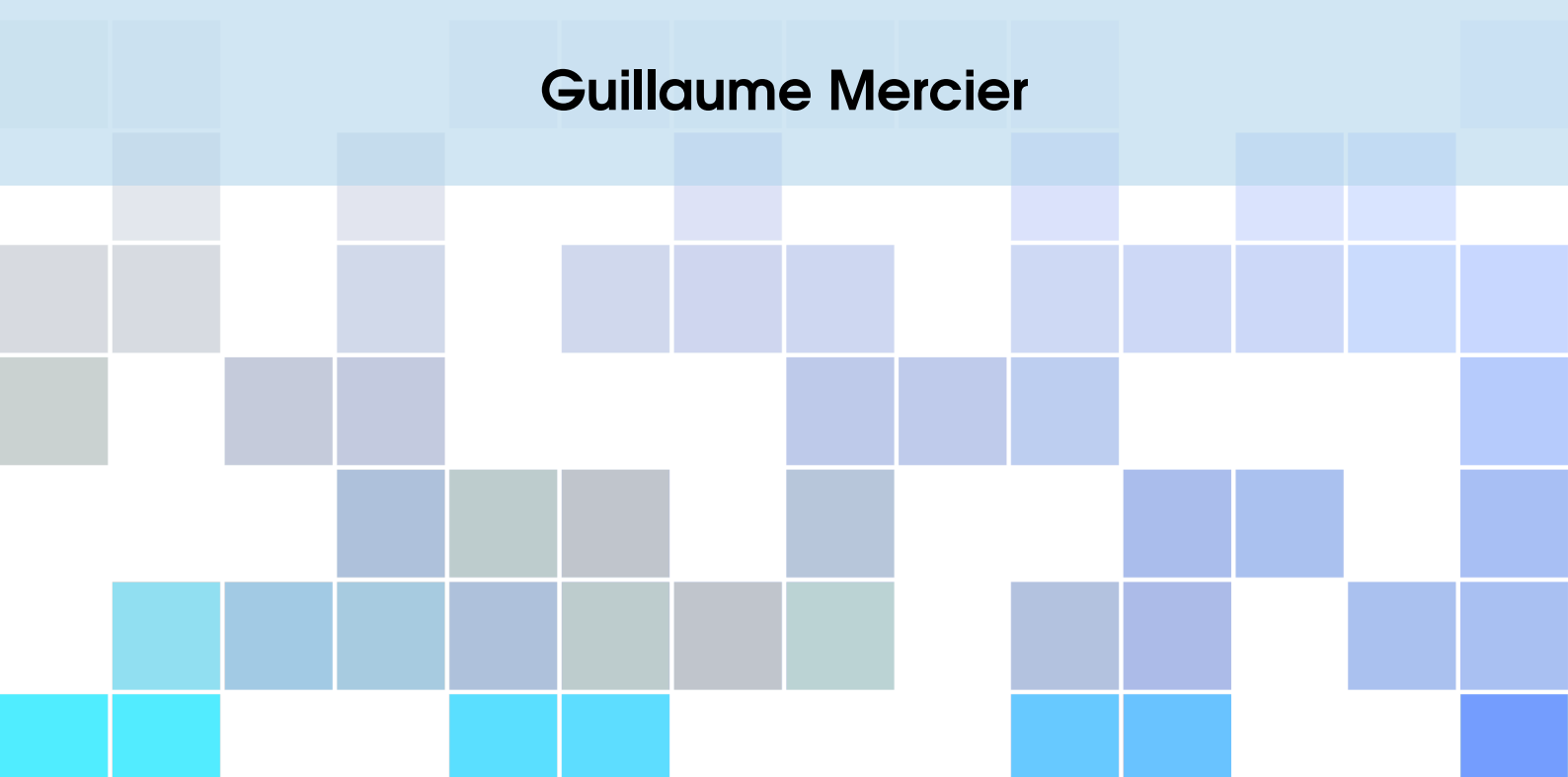


Éléments de programmation système (Télécom — ETE7-IFON1)

Part. Deux — Correction des exercices

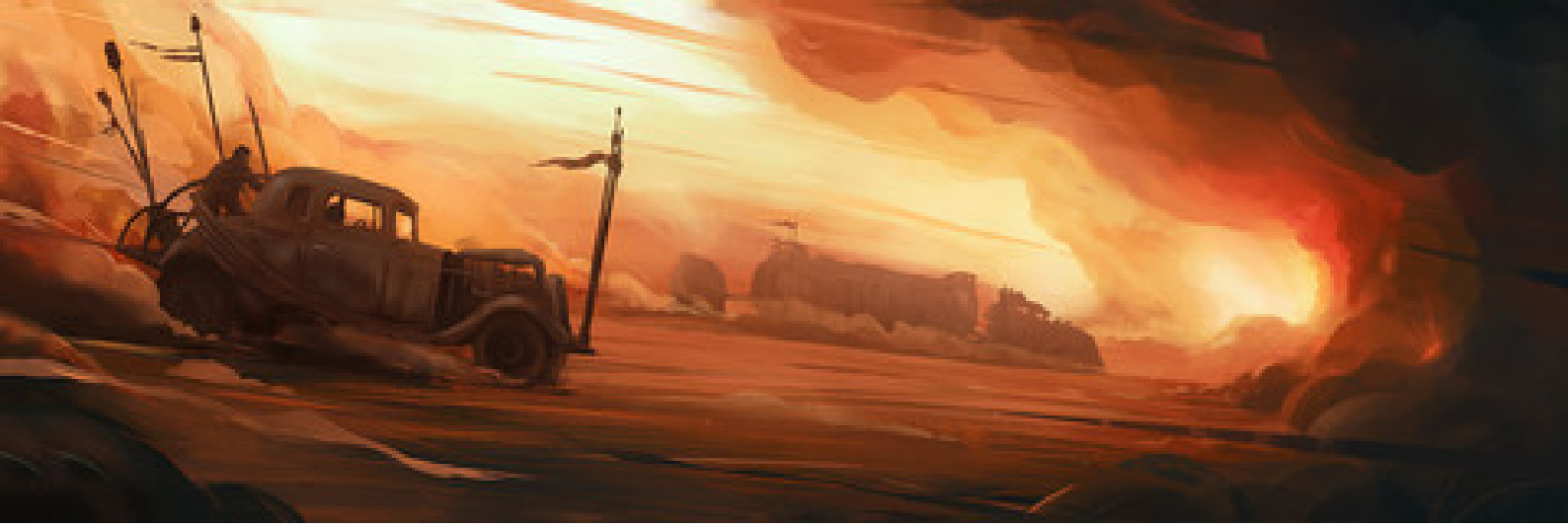
Année 2026 — 2027

Guillaume Mercier

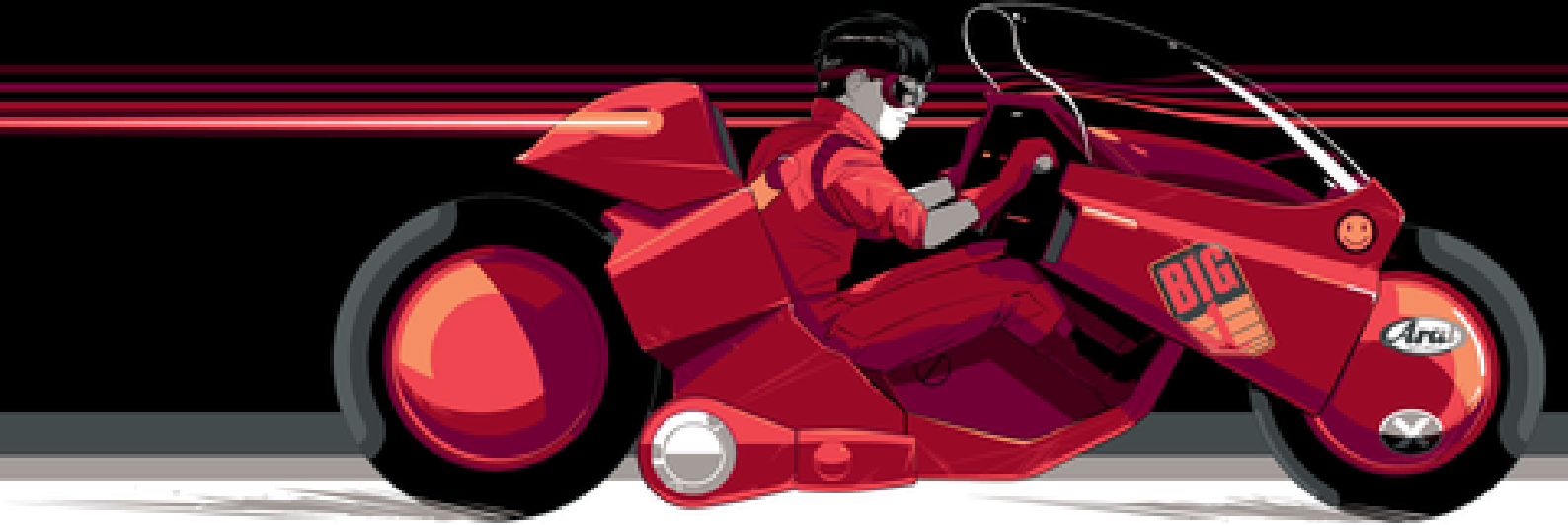


Copyright © 2026 Guillaume Mercier

La diffusion —même partielle— de ce document, quel qu'en soit le format (physique ou non) est interdite sans l'accord de son auteur. Les illustrations sont © de leurs auteurs respectifs.



1. Premier programme



2. Utilisation des fichiers

2.1 Notion de Système de fichiers

2.2 Structures du système pour la gestion des fichiers

Exercice 1 (★★)

Si la structure d'I-Nœud contient un tableau avec les adresses des blocs de données sur le disque et que la taille d'un fichier est susceptible de changer au cours du temps, comment peut-on faire pour stocker ce tableau sans que la taille de la structure d'I-Nœud elle-même ne change ?

Correction

Utiliser des indirections.

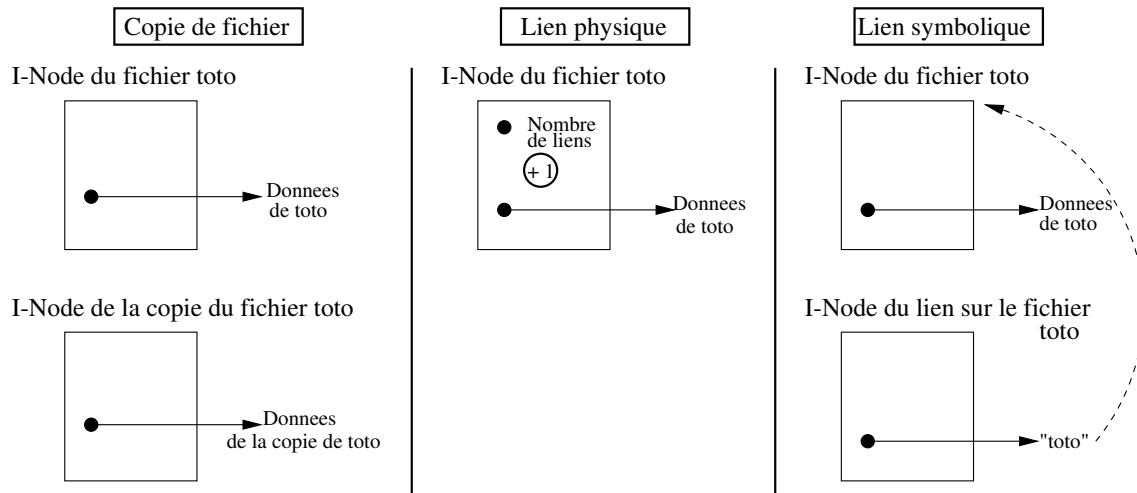
2.3 Nommage et partage de données

Exercice 2 (★★)

Quelles sont les différences, d'un point de vue des structures utilisées, entre copies de fichiers, liens physiques et liens symboliques ?

Correction

- Copie de fichier : un *nouvel* I-nœud est créé avec un pointeur sur une *copie* des données du fichier ;
- Lien physique : le nombre de liens sur le fichier est incrémenté. Pas de nouvelle structure d'I-nœud allouée ;
- Lien symbolique : un *nouvel* I-nœud est créé pour le lien symbolique qui est un type de fichier particulier (en tant que fichier, le lien doit donc posséder son propre I-nœud). Les données du lien sont en fait le nom du fichier sur lequel il pointe (une chaîne de caractères).



2.4 Interface pratique

Exercice 3 (☆☆☆)

Quelle est l'utilité du compte de référence de la structure de fichier ouvert puisqu'il semblerait qu'il ne puisse valoir qu'un ?

Correction

En fait, il peut valoir plus que 1, dans les cas de fork ou dup.

Exercice 4 (☆)

Créez un fichier dans votre répertoire de travail (avec une commande shell). Écrivez un programme qui ouvre ce fichier et affiche la valeur (de l'index) de descripteur de fichier. Cette valeur est-elle conforme à vos attentes ?

Correction

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <fcntl.h>
5
6 int main(int argc, char *argv[])
7 {
8     int fd = open("toto.txt", O_RDONLY);
9
10    fprintf(stdout, "valeur de l'index = %i\n", fd);
11
12    exit(EXIT_SUCCESS);
13 }
```

Exercice 5 (★)

Écrivez un programme qui ouvre un fichier en lecture seule et qui crée le fichier en mode lecture/écriture/exécution (uniquement pour vous). On supposera que ce fichier n'existe pas déjà.

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <fcntl.h>
5
6 int main(int argc, char *argv[])
7 {
8     int fd = open("toto.txt", O_RDONLY | O_CREAT, S_IRWXU );
9
10    fprintf(stdout, "valeur de l'index = %i\n", fd);
11
12    exit(EXIT_SUCCESS);
13 }
```

Exercice 6 (★)

Écrivez un programme qui écrit **un entier** dans un fichier (i.e. pas sous forme de chaîne de caractères). Dans un terminal, affichez le contenu de ce fichier (à l'aide d'une commande shell comme `cat`, `less` ou `more` et non pas d'un programme C).

— Que constatez-vous ?

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
9 {
10     int fd      = -1;
11     int valeur  = 33;
12     int ret     = -1;
13
14     fd = open("toto.txt", O_RDWR | O_CREAT, S_IRWXU);
15     assert( fd != -1);
16
17     ret = write(fd, &valeur, sizeof(valeur));
18     assert( ret != -1);
19
20     close(fd);
21     exit(EXIT_SUCCESS);
22 }
```

Exercice 7 (★)

Modifiez le programme précédent et rajoutez-y la lecture (et l'affichage) de l'entier que vous venez d'écrire dans le fichier.

Correction

On doit réutiliser le descripteur pour lire l'entier parce que la position courante après l'écriture est en fin de fichier. En fait, il serait plus efficace de modifier la position courante avec un appel à `lseek` mais cette fonction n'est pas connue au moment où cet exercice est fait.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
9 {
10     int fd      = -1;
```

```
11  int valeur  = 33;
12  int valeur2 = -1;
13  int ret     = -1;
14
15  fd = open("toto.txt", O_RDWR | O_CREAT, S_IRWXU);
16  assert(fd != -1);
17
18  ret = write(fd, &valeur, sizeof(valeur));
19  assert(ret != -1);
20
21  close(fd);
22
23  fd = open("toto.txt", O_RDONLY);
24  assert(fd != -1);
25
26  ret = read(fd, &valeur2, sizeof(valeur2));
27  assert(ret != -1);
28
29  fprintf(stdout, "Valeur lue = %i\n", valeur2);
30
31  close(fd);
32
33  exit(EXIT_SUCCESS);
34 }
```

Exercice 8 (★)

Écrivez un programme qui écrit n caractères dans un fichier. Les arguments du programme seront le nom du fichier et la valeur de n . Le i -ème caractère devra être écrit à la position 2^i dans le fichier, en supposant que la numérotation des caractères commence à 1.

Correction

Note : une solution alternative possible serait d'utiliser la fonction `pwrite` qui permet d'écrire directement à la position courante prise en dernier argument.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/stat.h>
6 #include <fcntl.h>
7 #include <assert.h>
8 #include <errno.h>
9
10 int main(int argc, char *argv[])
11 {
12     int    fd    = -1;
13     off_t  pos   = 2;
14     char   val   = 0;
15
16     assert(3 == argc);
17
18     fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, S_IRWXU);
19
20     for(int i = 1 ; i <= atoi(argv[2]) ; i++ , pos *= 2){
21         val = 'a';
22         lseek(fd, pos, SEEK_SET);
23         fprintf(stdout, "Write %c at pos %li\n", val, pos);
24         write(fd, &val, sizeof(char));
25     }
26
27     close(fd);
28
29     return 0;
30 }
```

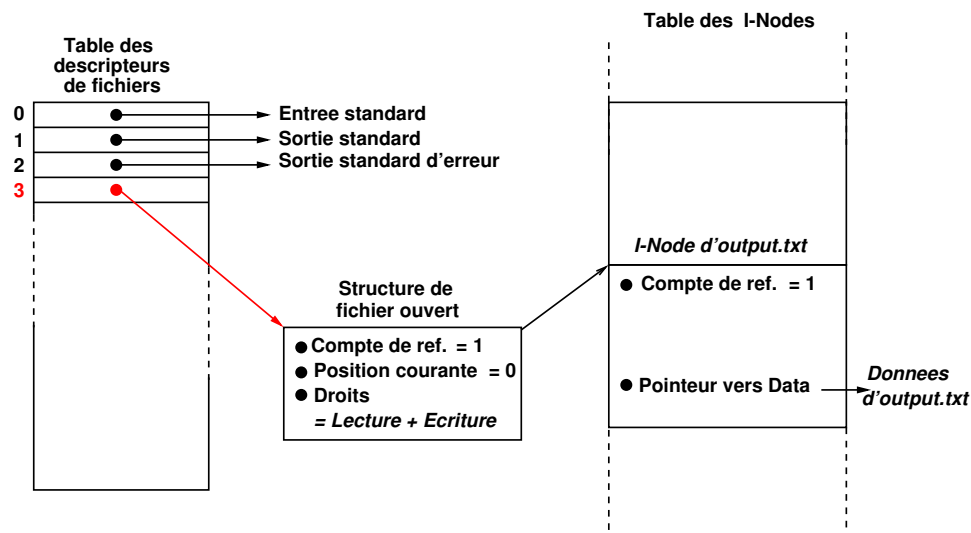
Exercice 9 (★)

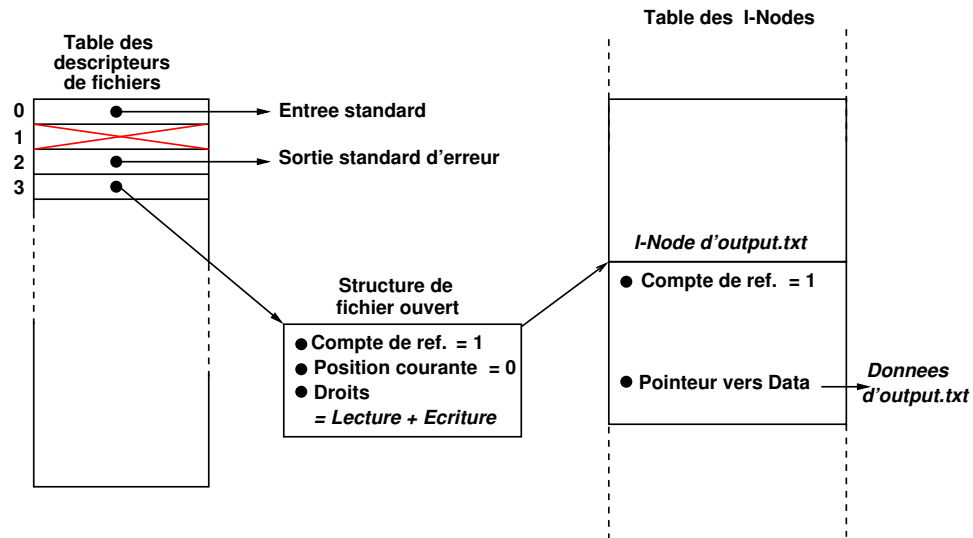
Écrivez un programme qui redirige tous ses appels à `printf` dans un fichier `output.txt` plutôt que sur sa sortie standard, sans modifier les appels à `printf` dans le programme d'origine.

Correction

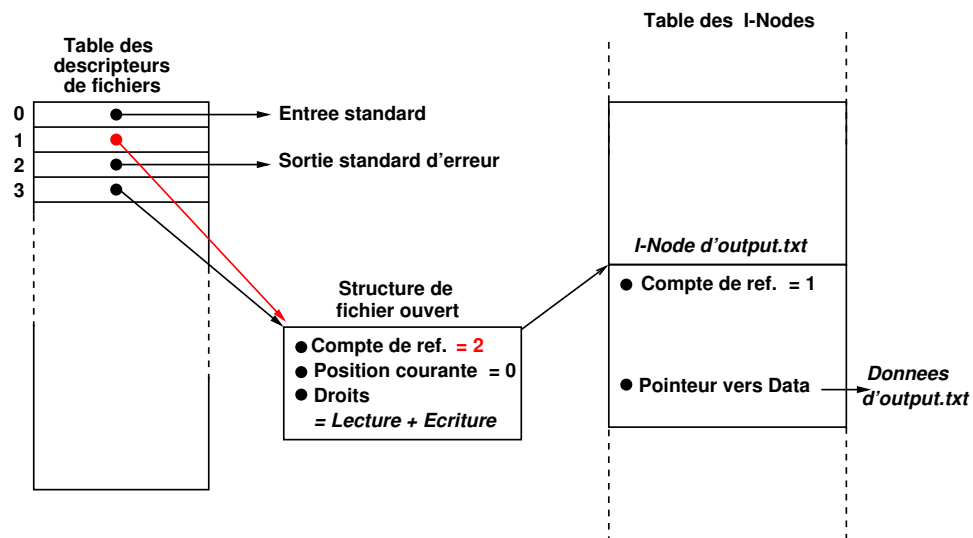
```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
9 {
10     int fd = -1;
11
12     fd = open("output.txt", O_RDWR | O_CREAT , S_IRWXU);
13     assert(fd != -1);
14
15     close(STDOUT_FILENO);
16     dup(fd);
17     close(fd);
18
19     printf("Hello World!\n");
20
21     exit(EXIT_SUCCESS);
22 }
```

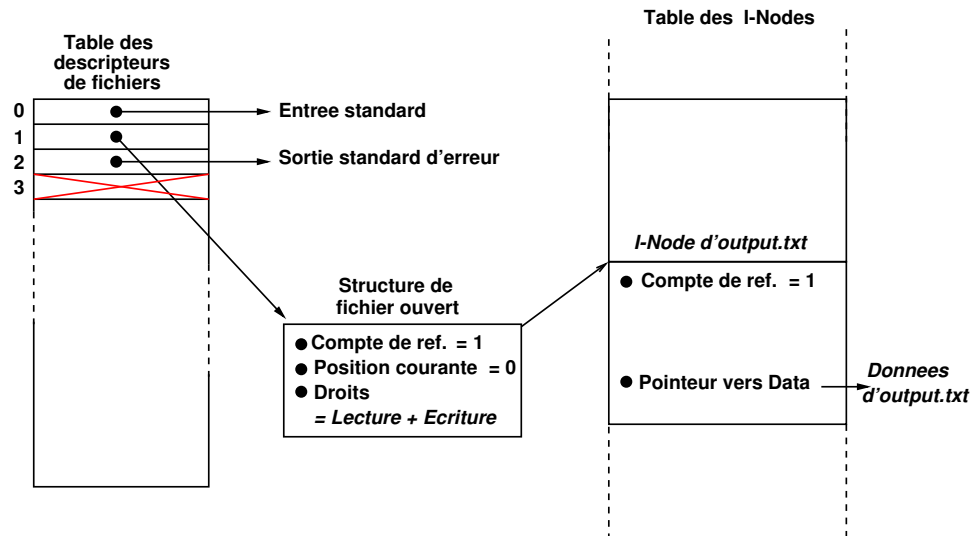




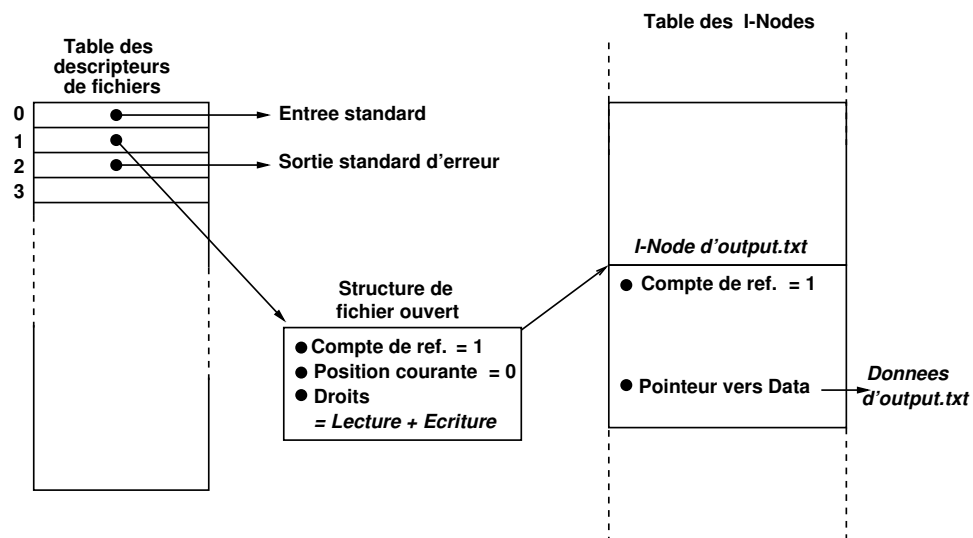
Exécution de la ligne 15



Exécution de la ligne 16



Exécution de la ligne 17



Situation finale

Exercice 10 (★)

Que se passera-t-il si un programme effectue les instructions suivantes ?

```

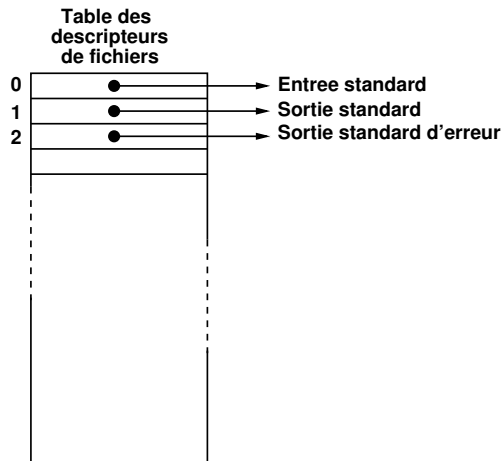
1 close (STDOUT_FILENO) ;
2 open ("toto.txt", O_RDWR) ;

```

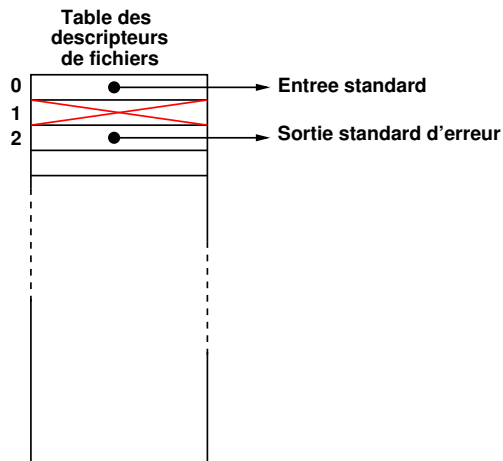
NB : On supposera que le fichier toto.txt existe.

Correction

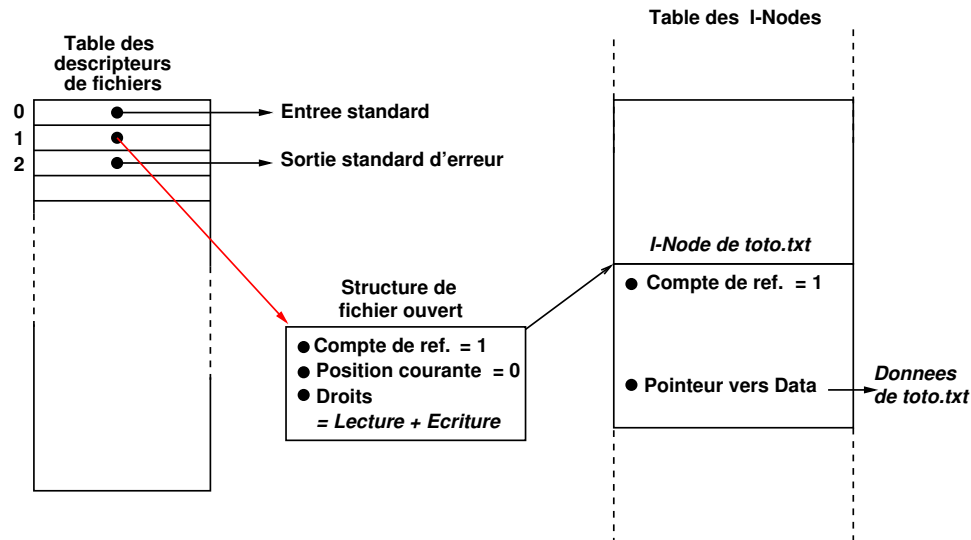
Ces deux lignes de code établissent une redirection de la sortie standard vers le fichier toto.txt. Fermer STDOUT_FILENO crée un « trou » dans la table des descripteurs de fichiers du processus appelant. Ledit « trou » est ensuite réutilisé lorsque un nouveau descripteur doit être alloué au moment de l'ouverture du fichier toto.txt.



Situation initiale



Exécution de la ligne 1



Exécution de la ligne 2

Exercice 11 (★)

Écrivez un programme qui ouvre un fichier avec la *fonction* `fopen` puis qui ouvre un fichier (le même ou un autre, peu importe) avec l'*appel-système* `open`. Affichez la valeur de l'index de descripteur associé.

- Que constatez-vous ?
- Que pouvez-vous en déduire ?

Correction

L'index du descripteur est 4, ce qui signifie qu'un autre descripteur a été alloué, pendant le `fopen` a priori.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
9 {
10     FILE *ptr = NULL;
11     int fd = -1;
12
13     ptr = fopen("output.txt", "r");
14     assert(ptr);
15
16     fd = open("output.txt", O_RDWR);
17     assert(fd != -1);

```

```

18
19     fprintf(stdout, "Valeur de l'index = %i\n", fd);
20
21     close(fd);
22     fclose(ptr);
23
24     exit(EXIT_SUCCESS);
25 }

```

Exercice 12 (★)

Écrivez un programme qui redirige tous ses appels à `fprintf(stdout, ...)` dans un fichier `output.txt` plutôt que sur sa sortie standard, sans modifier les appels à `fprintf` dans le programme d'origine et sans utiliser des (index de) descripteurs de fichiers ni la duplication.

Correction

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char *argv[])
5 {
6     FILE *fd = NULL;
7
8     fprintf(stdout, "Coucou - 1\n");
9
10    fd = freopen("output.txt", "w+", stdout);
11
12    fprintf(stdout, "Coucou - 2 \n");
13
14    fclose(fd);
15
16    fprintf(stdout, "Coucou - 3 \n");
17
18    exit(EXIT_SUCCESS);
19 }

```

2.5 Exercices

Exercice 13 (★★)

Écrivez un programme qui lit l'*intégralité* d'un fichier, caractère par caractère (i.e. octet par octet) et qui affiche le tout sur sa sortie standard d'*erreur*.

N.B L'utilisation de `stat` ou `fstat` n'est pas autorisée pour cet exercice.

Correction

Solutions possibles :

- Version 1 : tester le code de retour de `read` : 0 signifie que la fin de fichier est atteinte
- Version 2 : tester le caractère lu : EOF est le caractère de fin de fichier. Attention : ce caractère n'est pas toujours systématiquement disponible, il faut vérifier s'il est disponible pour la fonction utilisée. Par exemple, il existe pour `getc`, mais pas pour `read`...
- Version 3 : tester la fin de fichier avec la fonction `feof`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 #if VERSION == 1
9 int main(int argc, char *argv[])
10 {
11     int fd = -1;
12     char tmp;
13
14     fd = open("input.txt", O_RDONLY);
15
16     while(read(fd, &tmp, sizeof(char))) {
17         fprintf(stderr, "%c", tmp);
18     }
19
20     close(fd);
21     exit(EXIT_SUCCESS);
22 }
23
24 #elif VERSION == 2
25 int main(int argc, char *argv[])
26 {
27     FILE *file = NULL;
28     char tmp;
29
30     file = fopen("input.txt", "r");
31
32     // tmp = getc(file) est equivalent a
33     // fread(&tmp, sizeof(char), 1, file);
34     // getc permet de tester un caractere
35     // particulier, le caractere EOF (End-Of-File)
36
37     while( (tmp = getc(file)) != EOF) {
38         fprintf(stderr, "%c", tmp);
39     }
40
41     fclose(file);
42     exit(EXIT_SUCCESS);
43 }
44
45 #else // VERSION == 3
46 int main(int argc, char *argv[])
47 {
48     FILE *file = NULL;
49     char tmp;
```

```
50
51     file = fopen("input.txt", "r");
52
53     tmp = getc(file);
54     while( !feof(file)){
55         fprintf(stderr, "%c", tmp);
56         tmp = getc(file);
57     }
58
59     fclose(file);
60     exit(EXIT_SUCCESS);
61 }
62
63 #
64 #endif
```

Exercice 14 (★★)

On suppose l'existence d'un fichier texte contenant *un* mot par ligne avec plusieurs lignes, par exemple :

toto
titi
tata

Écrivez un programme qui :

1. ouvre ce fichier en lecture seule ;
2. compte le nombre de lignes dans le fichier ;
3. alloue un tableau dynamiquement pour contenir les différents mots (on suppose que leur taille fait au maximum `MAX_SIZE`). La taille de ce tableau doit correspondre *exactement* au nombre de mots (i.e. de lignes) dans le fichier ;
4. stocke les mots dans le tableau ;
5. affiche le contenu de ce tableau.

Correction

Pas de correction pour cet exercice qui sera réutilisé en projet Système & Réseaux.

2.6 Travaux Pratiques

Pas de correction pour cette partie.

Exercice 15 (★)

Écrivez une version en C de la commande `cat` en utilisant uniquement les fonctions standards d'entrées/sorties de la bibliothèque du C `fread` et `fwrite`, et en lisant et écrivant des blocs d'**un** caractère (oui, oui, un seul caractère ...).

Correction

Exercice 16 (★)

Écrivez une seconde version de cat en utilisant uniquement les appels-système read et write.

Correction**Exercice 17 (★)**

En utilisant la commande interne time du shell bash, comparez les performances des deux versions pour une lecture/écriture d'un fichier raisonnablement volumineux (i.e. de plus de 1 Mo).

Exemple :

```
$ time ./my_cat /lib/libc.a > tst
real    0m8.658s
user    0m2.920s
sys     0m5.740s
$
```

Interprétez les résultats obtenus. Quelles sont les commandes de plus bas niveau : read/write ou fread/fwrite ?

Correction**Exercice 18 (★)**

Essayez le programme suivant :

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    fwrite("Hello", sizeof(char), 6, stdout);
    write(STDOUT_FILENO, "World !", 7);
    exit(EXIT_SUCCESS);
}
```

L'observation est-elle en accord avec votre interprétation des résultats précédents ?

Correction



3. Du programme au processus

3.1 Exécution d'un programme

3.2 Identification d'un processus

Exercice 19 (★)

Écrivez un programme qui affiche le PID de son processus.

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
9 {
10     pid_t pid = -1;
11
12     pid = getpid();
13
14     fprintf(stdout, "Mon pid est = %i\n", pid);
15
16     exit(EXIT_SUCCESS);
17 }
```

3.3 Création de processus

Exercice 20 (★)

Écrivez un programme qui crée un processus-enfant et affiche ensuite une chaîne de caractères.

- Que constatez-vous ?
- Est-ce cohérent par rapport à ce que vous connaissez du fonctionnement de `fork` ?

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
9 {
10     fork();
11
12     fprintf(stdout, "Hello World!\n");
13
14     exit(EXIT_SUCCESS);
15 }
```

La chaîne "Hello World!" est affichée deux fois car tout le code situé après l'appel à `fork` est exécuté simultanément par le processus-parent et son enfant.

Exercice 21 (★★)

Reprenez le programme de l'exercice précédent et rajoutez un affichage de chaîne de caractères *avant* l'appel à `fork`.

- L'affichage est-il en accord avec vos attentes ? Pourquoi ?
- Que se passe-t-il lorsque il n'y a pas de caractère `\n` à la fin de la chaîne affichée par le `printf` placé avant le `fork` ?

Correction

Soit le code suivant :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
9 {
10     fprintf(stdout, "Hello World\n");
11
12     fork();
13 }
```

```
14 fprintf(stdout, "Hello World\n");
15
16 exit(EXIT_SUCCESS);
17 }
```

L’affichage est alors :

```
Hello World
Hello World
Hello World
```

Le premier "Hello World" est affiché par le parent et les deux suivants sont affichés par le parent et l’enfant (impossible de savoir dans quel ordre).

Si on enlève le caractère \n à la fin de la première chaîne, on aura alors comme affichage :

```
Hello WorldHello World
Hello WorldHello World
```

En effet, la première chaîne "Hello World" est écrite dans un buffer dans la mémoire du processus. Le contenu de ce buffer n’est pas affiché à l’écran car il n’y a pas de \n à la fin de la chaîne (ni d’appel à fflush après le printf). Au moment du fork, la mémoire du processus-parent est dupliquée dans le processus-enfant qui possède donc la chaîne "Hello World" dans son propre buffer. Au moment du second appel à printf, on écrit une nouvelle fois la chaîne "Hello World" dans le buffer puis on l’affiche (à cause du \n en fin de chaîne). Ceci se produit à la fois dans le parent et dans l’enfant, d’où l’affichage.

Exercice 22 (★★)

Écrivez un programme qui crée **exactement** 10 processus-enfants.

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
9 {
10     for(int index = 0 ; index < 10 ; index++){
11         pid_t child = fork();
12         if(0 == child){
13             break;
14         } else {
15             fprintf(stdout, "creation enfant %i\n", child);
16         }
17     }
18     exit(EXIT_SUCCESS);
19 }
```

Il est impératif que les processus-enfants sortent de la boucle (instruction `break`) sinon un nombre bien plus important de processus sera créé et non pas 10 (exactement).

3.4 Identifier le parent d'un processus

Exercice 23 (★)

Écrivez un programme qui crée un processus-enfant puis affiche le PID de ce dernier tandis que le processus-enfant affichera le PID de son parent. La chaîne à afficher sera donc différente selon que l'on se trouve dans le parent ou dans l'enfant.

Correction

On utilise le code de retour de `fork` pour savoir si on se trouve dans le processus-parent ou dans le processus-enfant :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
9 {
10     pid_t child = fork();
11
12     if(0 == child){
13         fprintf(stdout, "Enfant: pid parent = %i\n", getppid());
14     } else if (child > 0){
15         fprintf(stdout, "Parent: creation de l'enfant de pid
16             %i\n", child);
17     }
18
19     exit(EXIT_SUCCESS);
20 }
```

Exercice 24 (★)

Écrivez un programme qui affiche son PID puis crée un processus-enfant. Le processus-parent affiche ensuite le PID de son enfant tandis que son enfant affichera son propre PID puis celui de son parent. Vérifiez la cohérence de l'affichage.

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
```

```

9 {
10     pid_t child = -1;
11
12     fprintf(stdout, "Parent: je suis : %i\n", getpid());
13
14     child = fork();
15
16     if( child > 0){
17         fprintf(stdout, "Parent: Child : %i\n", child);
18     } else if (0 == child){
19         fprintf(stdout, "Enfant: je suis : %i\n", getpid());
20         fprintf(stdout, "Enfant: Parent : %i\n", getppid());
21     }
22
23     exit(EXIT_SUCCESS);
24 }

```

L'affichage est conforme à nos attentes~:

```

mercierg@Palamede$ ./Exercice_fork_affichage_pid_ppid
Parent: je suis : 520026
Parent: Child : 520027
Enfant: je suis : 520027
Enfant: Parent : 520026

```

Cependant, l'affichage peut apparaître incohérent, avec l'enfant qui affiche un pid pour son parent ne correspondant pas à ce qu'il devrait être, par exemple :

```

mercierg@Palamede$ ./Exercice_fork_affichage_pid_ppid
Parent: je suis : 520028
Parent: Child : 520029
Enfant: je suis : 520029
Enfant: Parent : 2510

```

L'explication se trouve en Section [3.7](#) (du volume 1).

3.5 Héritage inter-processus

Exercice 25 (★★)

Soit un processus qui ouvre un fichier régulier quelconque. Ce processus crée ensuite un processus-enfant à l'aide de `fork()`. Faites le schéma résumant l'état des structures de données : tables des descripteurs de fichiers, structures de fichiers ouverts et Table des I-Nœuds suite à ces opérations.

Correction

Soit la succession d'instructions suivante :

```

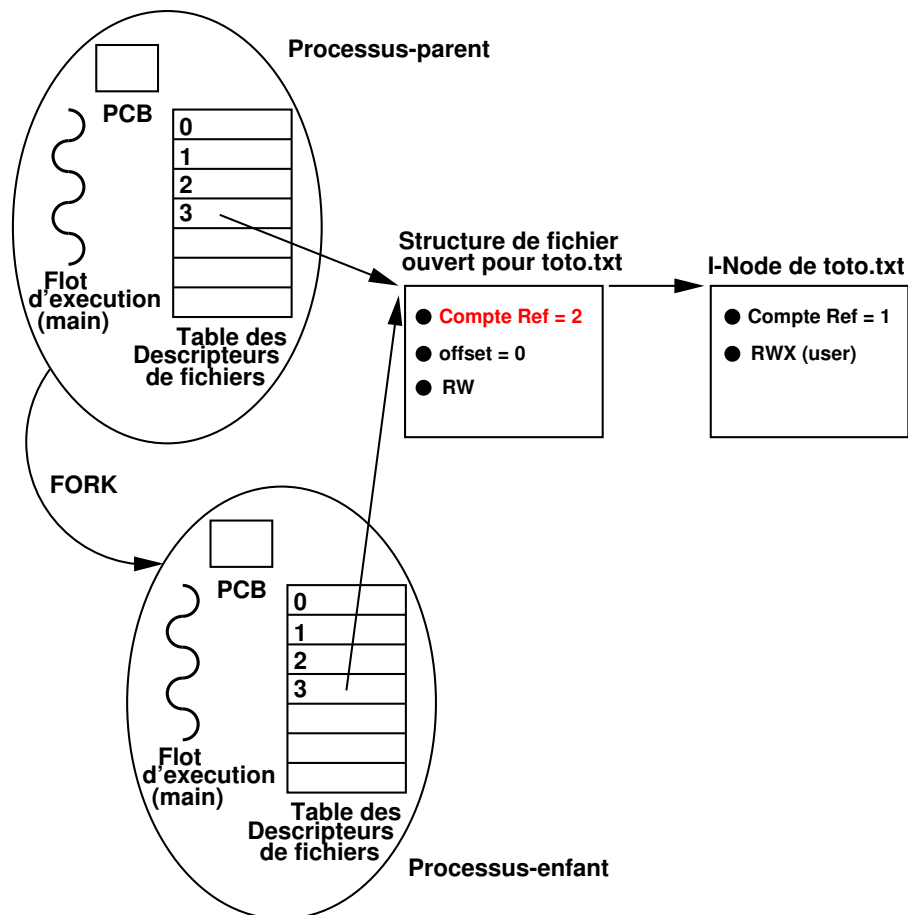
1 int main(int argc, char *argv[]){
2
3     int fd = open("toto.txt", O_RDWR | O_CREAT, S_IRWXU);
4
5     pid_t child = fork();

```

```

6
7  /* reste du code */
8
9  exit(EXIT_SUCCESS);
10 }
```

Voilà ce qui se passe au niveau des structures de données :



C'est le second cas où le compte de référence d'une structure de fichier ouvert peut être supérieur à 1 (le premier cas étant la duplication de descripteur).

3.6 *Modèle fork/join*

Exercice 26 (★)

Écrivez un programme qui :

1. crée un processus-enfant
2. effectue une petite boucle d'attente active, par exemple :

```
do{int toto = 0;
  while(++toto);
}while(0);
```
3. se synchronise avec cet enfant pour sa terminaison

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <sys/wait.h>
6 #include <errno.h>
7 #include <assert.h>
8
9 int main(int argc, char *argv[])
10 {
11     pid_t child = fork();
12
13     if( child > 0){ // code du parent
14         short int toto = 0;
15         pid_t fini = -1;
16
17         fprintf(stdout, "Parent : je suis : %i | pid enfant %i\n",
18             getpid(), child);
19
20         while(++toto); // attente active
21
22         fini = wait(NULL);
23
24         fprintf(stdout, "Parent : l'enfant %i est fini\n", fini);
25     }
26
27     exit(EXIT_SUCCESS);
28 }
```

Exercice 27 (★)

Reprenez le programme précédent et affichez dans le parent le PID de l'enfant terminé, ainsi que son code de retour. Le processus-enfant affichera son PID, celui de son parent puis attendra un petit peu avant d'appeler `exit`. Vous donnerez une valeur de retour bien identifiable pour l'enfant (i.e ni 0, 1 ou -1).

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <sys/wait.h>
6 #include <errno.h>
7 #include <assert.h>
8
9 int main(int argc, char *argv[])
10 {
11     pid_t child = fork();
12
13     if(0 == child){ // code de l'enfant
14         short int toto = 0;
15
16         fprintf(stdout,"Enfant : Je suis %i\n",getpid());
17         fprintf(stdout,"Enfant : pid parent %i\n",getppid());
18
19         while(++toto);
20
21         exit(33);
22     } else if (child > 0){ // code du parent
23         pid_t fini = -1;
24         int status = 0;
25
26         fprintf(stdout,"Parent : creation enfant %i\n",child);
27
28         fini = wait(&status);
29         fprintf(stdout,
30             "Parent : le processus %i est fini, valeur de
31                 retour = %i\n",
32                 fini,
33                 WEXITSTATUS(status));
34     }
35     exit(EXIT_SUCCESS);
36 }
```

Exercice 28 (★★)

Écrivez un programme qui crée quelques processus-enfants, puis fait une boucle d'attente active et enfin les attend dans l'ordre décroissant de leur valeur de PID. Les processus-enfants feront quant à eux une boucle d'attente (active ou passive) de durée variable selon leur PID.

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/types.h>
4  #include <sys/stat.h>
5  #include <fcntl.h>
6  #include <string.h>
7  #include <errno.h>
8  #include <unistd.h>
9  #include <sys/wait.h>
10
11 #define NUM_ENFANT 5
12
13 int main(int argc, char **argv)
14 {
15     pid_t child[NUM_ENFANT];
16     int toto = 0;
17
18     for(int idx = 0 ; idx < NUM_ENFANT ; idx++){
19         child[idx] = fork();
20
21         if( 0 == child[idx]){
22             pid_t pid = getpid();
23             usleep(pid+1000);
24             exit(33+idx);
25         }
26         else{
27             fprintf(stdout, "==== Creation de l'enfant :
28                 %i\n", child[idx]);
29         }
30     }
31
32     while(++toto);
33
34     for(int idx = NUM_ENFANT - 1 ; idx >= 0 ; idx--){
35         pid_t child2 = waitpid(child[idx], NULL, 0);
36         fprintf(stdout, "Enfant %i termine \n", child2);
37     }
38
39     return EXIT_SUCCESS;
40 }
```

3.7 Terminaison d'un processus

Exercice 29 (☆☆☆)

Pouvez-vous donner un type de programme qui ne fonctionnera pas correctement (voire pas du tout) à cause d'un nombre de zombies trop important ?

Correction

Un processus-serveur qui crée des processus pour traiter les requêtes des clients et qui ne récupère pas la terminaison de ses enfants.

Exercice 30 (☆☆)

Écrivez un programme qui crée un processus-enfant et qui se termine immédiatement. Le processus-enfant devra s'endormir pendant quelques secondes puis afficher le PID de son parent.

- Exécutez ce programme plusieurs fois. Qu'observez-vous ?
- Est-ce cohérent ?

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <errno.h>
6 #include <assert.h>
7
8 int main(int argc, char *argv[])
9 {
10     pid_t child = -1;
11
12     fprintf(stdout, "Parent: je suis : %i\n", getpid());
13
14     child = fork();
15
16     if( child > 0){
17         fprintf(stdout, "Parent: Child : %i\n", child);
18     } else if (0 == child){
19         fprintf(stdout, "Enfant: je suis : %i\n", getpid());
20         sleep(3);
21         fprintf(stdout, "Enfant: Parent : %i\n", getppid());
22     }
23
24     exit(EXIT_SUCCESS);
25 }
```

Le pid affiché ne correspond pas forcément à celui du processus-parent. Adoption des orphelins-zombies par init (1) ou par systemd.

Pour aller plus loin : Retour sur la synchronisation parent-enfant

3.8 États d'un processus

Exercice 31 (★)

Reprenez un des exercices précédents avec de l'attente active et remplacez-la par un appel à `sleep`. Qu'observez-vous ?

Correction

Ça ne change rien a priori, sauf au niveau des performances car le processus passe de l'état actif à l'état endormi/bloqué et n'occupe pas le processeur.

3.9 Recouvrement

Exercice 32 (★)

Créez un premier programme basique (qui effectue juste un appel à `printf` par exemple) puis créez un second programme qui effectue un recouvrement avec le code de ce premier programme.

Correction

Soit le code suivant (`hello_world.c`, compilé en exécutable `hello_world`) :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char *argv[])
5 {
6     printf("Hello World!\n");
7
8     exit(EXIT_SUCCESS);
9 }
```

Le recouvrement se fait de la façon suivante :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <sys/wait.h>
6 #include <errno.h>
7 #include <assert.h>
8
9 int main(int argc, char *argv[])
10 {
11     char *my_argv[] = {"hello_world", NULL};
12
13     execv("hello_world", my_argv);
14
15     exit(EXIT_SUCCESS);
16 }
```

Exercice 33 (★★)

Reprenez le programme de l'exercice précédent, mais le recouvrement devra être effectué par un processus-enfant créé dans le second programme.

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <sys/wait.h>
6 #include <errno.h>
7 #include <assert.h>
8
9 int main(int argc, char *argv[])
10 {
11     pid_t child = fork();
12
13     if(0 == child){ // Code de l'enfant
14
15         char *my_argv[] = {"hello_world", NULL};
16
17         execv("hello_world", my_argv); // recouvrement
18
19     } else if ( child > 0) { // Code du parent
20
21         wait(NULL); //synchronisation
22
23     }
24
25     exit(EXIT_SUCCESS);
26 }
```

Exercice 34 (★★)

Reprenez l'exercice précédent, mais le programme utilisé pour le recouvrement effectuera la somme de ses paramètres (dont on suppose que ses arguments sont uniquement des entiers).

Correction

Soit le code du programme `somme.c`, compilé en un exécutable appelé `somme` :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char *argv[])
5 {
6     int somme = 0;
7
8     while(*++argv)
9         somme += atoi(*argv);
```

```

10
11     fprintf(stdout, "Somme = %i\n", somme);
12
13     exit(EXIT_SUCCESS);
14 }

```

Alors le recouvrement se fera de la façon suivante :

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <sys/wait.h>
6  #include <errno.h>
7  #include <assert.h>
8
9  int main(int argc, char *argv[])
10 {
11     pid_t child = fork();
12
13     if(0 == child){ //Code de l'enfant
14
15         //Tableau des arguments
16         char *my_argv[] = {"somme", "1", "2", "3", NULL};
17
18         execv("somme", my_argv); // recouvrement
19
20     } else if (child > 0) { // Code du parent
21
22         wait(NULL); // synchronisation
23
24     }
25
26     exit(EXIT_SUCCESS);
27 }

```

L'appel produit bien le résultat attendu :

```

mercierng@Palamede:$ ./Exercice_recouvrement_somme
Somme = 6

```

Exercice 35 (★★)

Écrivez un programme qui ouvre un fichier quelconque et affiche la valeur (de l'index) du descripteur associé. Exécutez ce programme :

1. en le lançant directement depuis un terminal
 2. en créant un processus-enfant qui ouvre d'abord un fichier (le même ou un autre, peu importe) puis qui effectue un recouvrement (via `exec`) avec le premier programme.
- Que constatez-vous ?
- L'affichage est-il cohérent ?

Correction

Pour cet exercice on peut exécuter le code de l'exercice 4 (ou 5) (ouverture d'un fichier et affichage de l'index de descripteur associé).

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <sys/wait.h>
6 #include <errno.h>
7 #include <assert.h>
8
9 int main(int argc, char *argv[])
10 {
11     pid_t child = fork();
12
13     if(0 == child){ // Code de l'enfant
14
15         //Ouverture du fichier toto.txt, dont on suppose
16         //l'existence
17         int fd = open("toto.txt", O_RDONLY);
18         fprintf(stdout, "Valeur de l'index de descripteur =
19         %i\n", fd);
20         fflush(stdout);
21
22         //Tableau d'arguments
23         //On reexecute le code de l'exercice 4 (ouverture de
24         //fichier
25         // et affichage de l'index de descripteur
26         char *my_argv[] = {"Exercice_ouverture_fichier", NULL};
27
28         execv("Exercice_ouverture_fichier", my_argv);
29         //recouvrement
30
31     } else if ( child > 0) { // Code du parent
32
33         wait(NULL); // synchronisation
34     }
35
36     exit(EXIT_SUCCESS);
37 }
```

L'affichage est le suivant :

```
mercierg@Palamede:$ ./Exercice_fork_fichier
Valeur de l'index de descripteur = 3
valeur de l'index = 4
```

Comme un fichier a été ouvert par le processus, l'index est 3. Ensuite, on effectue un recouvrement du processus par un autre exécutable qui ouvre également un fichier. Mais comme le descripteur du

fichier précédent est conservé à travers le recouvrement, le nouvel index est 4.

Exercice 36 (★)

Reprenez l'exercice précédent en positionnant l'option `O_CLOEXEC` au moment de l'ouverture du fichier avant d'effectuer le recouvrement (ie. l'appel à `exec`). Que constatez-vous ?

Correction

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <sys/wait.h>
6 #include <errno.h>
7 #include <assert.h>
8
9 int main(int argc, char *argv[])
10 {
11     pid_t child = fork();
12
13     if(0 == child){ // Code de l'enfant
14
15         //Ouverture du fichier en mode CLOEXEC
16         int fd = open("toto.txt", O_RDONLY | O_CLOEXEC);
17         fprintf(stdout, "Valeur de l'index de descripteur =
18             %i\n", fd);
19         fflush(stdout);
20
21         //Tableau d'arguments
22         char *my_argv[] = {"Exercice_ouverture_fichier", NULL};
23         execv("Exercice_ouverture_fichier", my_argv);
24         //recouvrement
25
26     } else if (child > 0) { // Code du parent
27
28         wait(NULL); //synchronisation
29
30     }
31
32     exit(EXIT_SUCCESS);
33 }
```

L'affichage est le suivant :

```
mercierg@Palamede:$ ./Exercice_fork_fichier
Valeur de l'index de descripteur = 3
valeur de l'index = 3
```

Comme un fichier a été ouvert par le processus, l'index est 3. Ensuite, on effectue un recouvrement du processus par un autre exécutable qui ouvre également un fichier mais avec l'option `O_CLOEXEC`. Ce descripteur de fichier est donc fermé au moment du recouvrement et le nouvel index est donc

encore 3.

3.10 Pour aller plus loin : exécution de commandes shell dans un programme



4. Communication inter-flots



5. Interlude non-déterministe



6. Notification d'événements : les signaux



7. Splendeur et tourments du multithreading



8. Synchronisation inter-flots